Algorithms: Design and Analysis (Part I)

Mihir Rao

Contents

1 Introduction: Integer Multiplication and Merge Sort **2**

1.1 Integer multiplication and the cost of a step 2

1.2 Recursive multiplication and Karatsuba's idea 2

1.3 Divide and conquer through sorting 3

1.4 Merging two sorted arrays 4

1.5 Counting work in one merge 5

1.6 The recursion-tree analysis 5

1.7 Three principles for algorithm analysis 7

Lecture 1: Introduction: Integer Multiplication and Merge Sort

An algorithm specifies how to turn an input into a required output. Its design asks which steps to use; its analysis asks whether those steps are correct and how much work they require. Better algorithms can make a task feasible even when faster hardware alone would not. Algorithmic thinking also gives a way to study processes such as markets and evolution.

Two familiar tasks, multiplying integers and sorting numbers, introduce a recurring question: can we improve on the obvious method?

1.1 Integer multiplication and the cost of a step

The input is two nonnegative integers x and y, each represented with n decimal digits; the required output is xy. Here a primitive operation adds or multiplies two single digits. Multiplying two arbitrarily long integers is therefore not one primitive operation.

The grade-school method forms one partial product for each digit of y, then adds the shifted rows. For the given example,

$$\begin{array}{r} 5678 \\ \times 1234 \\ \hline 22712 \\ 170340 \\ 1135600 \\ 5678000 \\ \hline 7006652 \end{array}$$

There are n rows, each involving work proportional to n, including digit multiplication and carrying. The final additions also take at most a constant times n^2 work. Thus the total grows quadratically with the number of digits. Doubling the digit count roughly quadruples this work.

1.2 Recursive multiplication and Karatsuba's idea

Split each input into a high half and a low half. With $m = n/2$ for an even digit count, write

$$x = 10^m a + b, \qquad y = 10^m c + d,$$

where each block uses at most m digits, allowing leading zeros. Expanding the product gives

$$xy = 10^{2m} ac + 10^m (ad + bc) + bd. \tag{1.1}$$

This identity reduces the original task to products of smaller integers. A direct recursive method computes ac, ad, bc, bd separately, then combines them using equation (1.1). Small inputs are handled directly.

Karatsuba multiplication reuses information to remove one of these four recursive products. Compute

$$u = ac, \qquad v = bd, \qquad w = (a + b)(c + d).$$

Since $w = ac + ad + bc + bd$, the middle coefficient is $w - u - v = ad + bc$. Therefore

$$\boxed{xy = 10^{2m}u + 10^{m}(w - u - v) + v.}$$

There are three recursive multiplications, plus additions, subtractions, and decimal shifts. Sums such as $a + b$ can have an extra carry digit; a complete implementation must preserve it. The smaller number of recursive products is the reason to investigate this method further, rather than counting a recursive call as a single inexpensive step.

Example: For $x = 5678$ and $y = 1234$, take $a = 56$, $b = 78$, $c = 12$, $d = 34$. Then

$$u = 56 \cdot 12 = 672, \qquad v = 78 \cdot 34 = 2652,$$
$$w = 134 \cdot 46 = 6164, \quad w - u - v = 2840.$$

Recombining gives

$$10^4(672) + 10^2(2840) + 2652 = 7006652,$$

which agrees with the grade-school calculation.

1.3 Divide and conquer through sorting

A divide and conquer algorithm splits a task into smaller subproblems, solves them recursively, and combines their answers. The combination step matters: reducing the input size is useful only if the work of solving and combining the subproblems is controlled.

Definition 1.1 [Sorting problem]. Given an array of n distinct numbers, return the same numbers in increasing order. Correctness requires both ordering and preservation of the input elements.

Merge sort splits the array into two halves, sorts each half recursively, and merges the sorted halves. Arrays of length zero or one are already sorted.

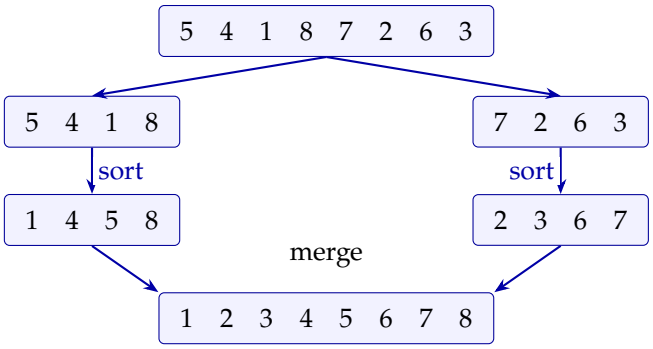


Figure 1: Merge sort on the supplied eight-element array. The two recursive calls return sorted halves; the final merge combines them in increasing order.

Algorithm 1.1: Merge sort

Input: Array X of length n, with distinct entries.
Output: The entries of X in increasing order.

```
if n ≤ 1 then
    return X
m ← ⌊n/2⌋
A ← MergeSort(X[1 : m])
B ← MergeSort(X[m + 1 : n])
return Merge(A, B)
```

Both recursive arrays have length smaller than n when $n \geq 2$, so the procedure eventually reaches a base case. The floor handles odd lengths without discarding an element.

1.4 Merging two sorted arrays

The smallest unused element of a sorted array is its first unused element. To find the next output element, it is therefore enough to compare the two current front elements. After choosing one, advance only its pointer. If an array is exhausted, copy the remaining elements of the other array.

Algorithm 1.2: Merge two sorted arrays

Input: Increasing arrays $A[1 : p]$ and $B[1 : q]$, with distinct entries overall.
Output: An increasing array C containing every input entry once.

```
i ← 1; j ← 1; C ← an empty array
while i ≤ p and j ≤ q do
    if A[i] < B[j] then
        Append A[i] to C; i ← i + 1
    else
        Append B[j] to C; j ← j + 1
Append the remaining suffix A[i : p], if nonempty, to C
Append the remaining suffix B[j : q], if nonempty, to C
return C
```

Lemma 1.2 [Correctness of merging]. *If A and B are sorted, the merge procedure returns their entries in increasing order, with no loss or repetition.*

Proof. The idea is to maintain that the output contains exactly the smallest entries processed so far. Initially it is empty. While both arrays have unused entries, the smaller front element is the smallest unused element of their union, because neither sorted suffix contains a smaller element behind its front. Appending that element preserves the ordering and moves exactly one pointer. When one array is exhausted, the remaining sorted suffix contains all remaining elements, each at least as large as the last element appended. Copying it finishes the output. Every element is copied once, so the result has the required order and contents. □

Proposition 1.3 [Correctness of merge sort]. *Merge sort returns the input entries in increasing order.*

Proof. Use induction on the array length. Lengths zero and one are base cases. For $n \geq 2$, each half is shorter than the input, so the induction hypothesis guarantees that the recursive calls sort both halves correctly. The merge lemma then orders their union and preserves all entries. This proves the claim for length n. □

1.5 Counting work in one merge

Let $m = p + q$ be the total number of input entries. Each iteration copies one entry, and the suffix copies account for exactly the remaining entries. Thus merging takes work proportional to m.

For a coarse operation count, use the simplified loop accounting: two pointer initializations and at most four elementary steps for each output position. This gives

$$\text{MergeCost}(m) \leq 4m + 2 \leq 6m \qquad (m \geq 1).$$

The constant depends on which bookkeeping steps are charged. Completing the end cases in actual code can change it, but does not change the linear growth. This bound is a mathematical accounting model, not a count of machine instructions.

1.6 The recursion-tree analysis

Each merge is linear in its own input size, but there are many merges. The useful observation is that the total input size across any one level of recursion is n.

Assume first that $n = 2^h$, so all splits are even. A recursion tree has one node per recursive call. At depth j there are 2^j subproblems, each of size $n/2^j$. The leaves have size one, so they occur at depth $h = \log_2 n$. There are $h + 1$ levels, counting the root and the leaves.

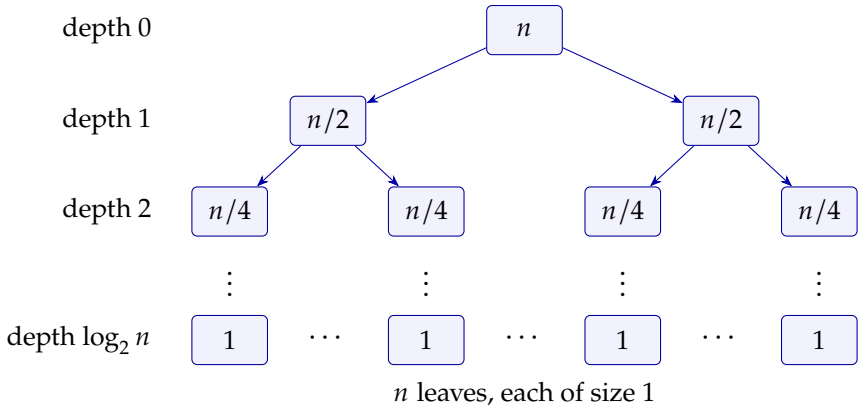


Figure 2: Schematic recursion tree for n a power of two. The number of subproblems doubles as their sizes halve. Intermediate levels are omitted.

Theorem 1.4 [Merge-sort running time]. *In the coarse operation model above, merge sort on $n = 2^h \geq 1$ entries uses at most $6n \log_2 n + 6n$ charged operations, including a constant charge for each leaf.*

Proof. Bound the work one level at a time. Charge at most $6m$ for a node of size m, with leaf work bounded by 6. The charge at depth j is at most

$$\underbrace{2^j}_{\text{number of nodes}} \quad \underbrace{6\left(\frac{n}{2^j}\right)}_{\text{work per node}} = 6n.$$

There are $\log_2 n + 1$ levels. Summing their charges gives

$$\sum_{j=0}^{\log_2 n} 6n = 6n(\log_2 n + 1) = 6n \log_2 n + 6n.$$

The leaf charge is included, even though a leaf performs no merge. □

For a general positive n, balanced splitting has at most $\lceil \log_2 n \rceil + 1$ levels. Nodes at any fixed depth cover disjoint portions of the input, whose total size is at most n. The same reasoning gives a bound proportional to $n \log n$. The power-of-two assumption makes the displayed level counts exact; it is not required by the algorithm.

1.7 Three principles for algorithm analysis

1. Worst-case analysis gives a running-time bound for every input of a given size. It is useful for a general-purpose routine whose future inputs are unknown. An average-case guarantee needs a model for which inputs are likely; a benchmark measures selected instances.
2. Suppress constant factors and lower-order terms when comparing growth rates. Constants depend on implementation and computing hardware; retaining all of them can obscure the main design difference.
3. Asymptotic analysis studies how the cost grows as the input becomes large. A smaller growth rate need not give a smaller cost on every small input.

For example, compare the illustrative costs

$$f(n) = 6n \log_2 n + 6n, \qquad g(n) = \tfrac{1}{2}n^2.$$

The linearithmic expression eventually becomes much smaller, although the quadratic expression is smaller over part of the range of small n. These are cost formulas, not measured runtimes.

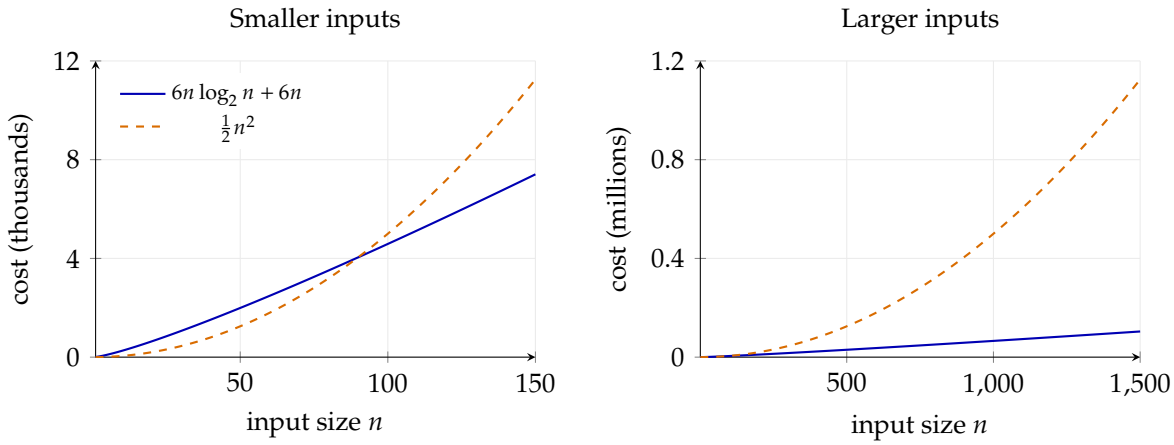


Figure 3: The supplied illustrative cost functions at two input scales. The relative advantage of $6n \log_2 n + 6n$ grows with the input size. The vertical axes use different units.

The aim is a worst-case cost that grows slowly with input size, often as close to linear as the task permits. Merge sort illustrates why this viewpoint helps: its modest per-level work is repeated only logarithmically many times.

Revision. Specify the input, output, and primitive operations before counting work. Karatsuba obtains the cross term from three recursive products. Merge sort uses a linear merge at each node; the total work at each recursion depth is linear, and there are logarithmically many depths. Correctness follows by combining a merge invariant with induction.
Common mistakes: charging one step for a large-integer product; forgetting a carry digit in $a + b$; reading past an exhausted merge input; counting $\log_2 n$ levels while also including the leaves; or treating an asymptotic comparison as a timing claim for every input size.
Check yourself: How does $(a + b)(c + d)$ eliminate one multiplication? Why is the smaller front element safe to output? Why is the work per tree level independent of its depth? Which assumption makes the level counts exact, and which parts of the algorithm work without it?

Source. Tim Roughgarden, *Algorithms: Design and Analysis (Part I)*, Introduction slide group, from the instructor's course collection. Eight annotated PDF decks, 68 pages in total: Why Study Algorithms? (pp. 1–7), Integer Multiplication (pp. 1–4), Karatsuba Multiplication (pp. 1–4), About the Course (pp. 1–27), Merge Sort: Overview (pp. 1–4), Pseudocode (pp. 1–7), Analysis (pp. 1–8), and Guiding Principles (pp. 1–7). Original PDFs and retrieval provenance are retained privately in `sources/`. The decks were retrieved from a pinned GitHub mirror; equivalence to the current official archive remains unverified. Repeated builds are consolidated and administration omitted. Diagrams are original redraws. Base cases, merge exhaustion handling, the merge invariant, inductive correctness argument, and the non-power-of-two level bound complete the supplied procedures and claims. Operation constants use the stated coarse accounting model.